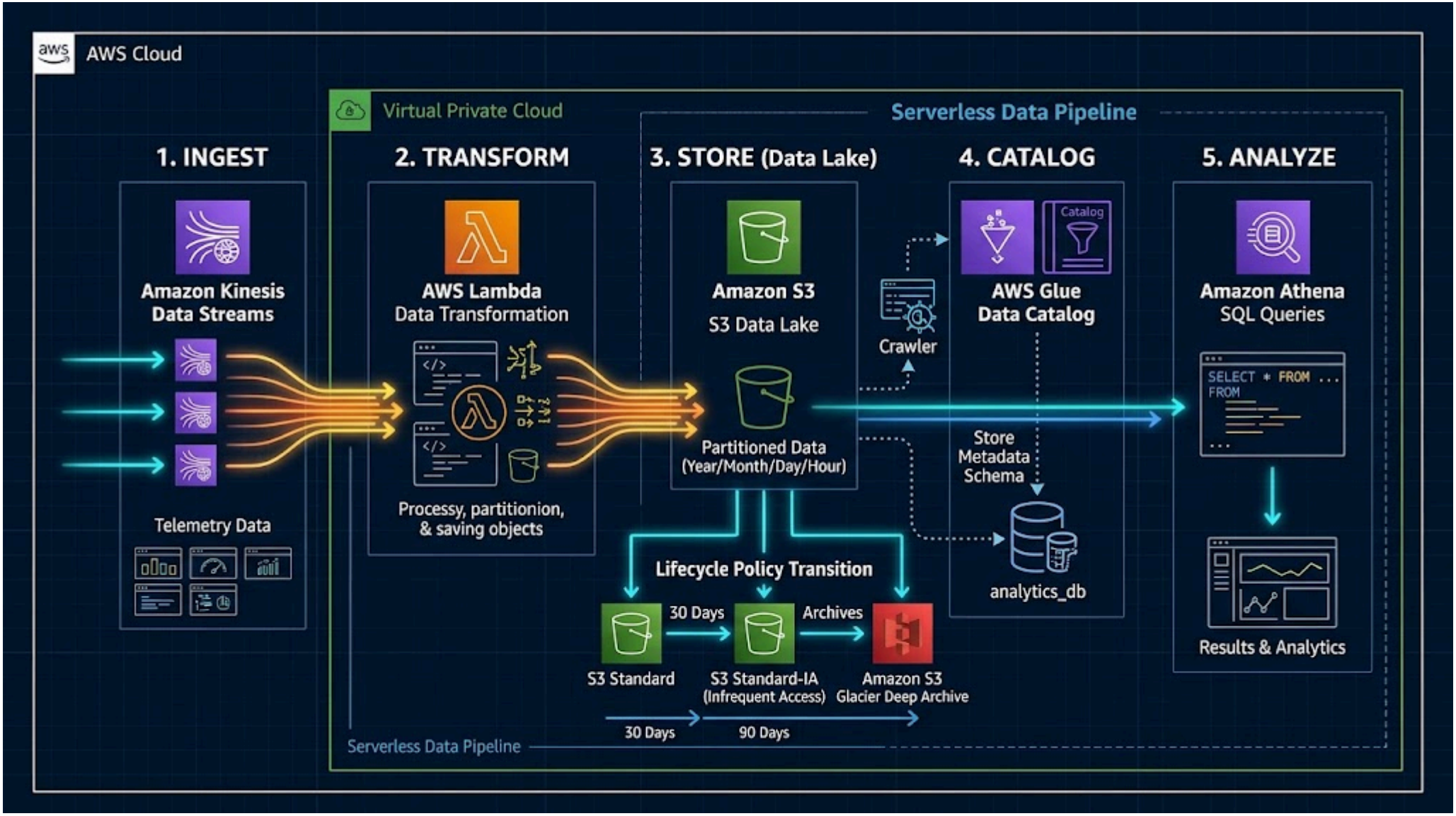
10th Lab: Real-Time Serverless Data Ingestion, Analytics & Tiered Storage Lifecycle

Estimated Duration: 1.5 to 2 Hours

Focus Areas: Amazon Kinesis Data Streams, AWS Lambda, Amazon S3 Lifecycle Rules, Amazon Athena, AWS Glue Data Catalog, AWS KMS Encryption.



Architecture Overview & Blueprint

This hands-on lab introduces a critical enterprise data engineering pattern tested on the AWS Solutions Architect Associate (SAA-C03) exam and widely used in AI/cloud infrastructure: building an event-driven, real-time data ingestion pipeline with automated schema cataloging, SQL analytics, and lifecycle cost optimization.

Component	AWS Service	Architectural Purpose
Ingestion Tier	Amazon Kinesis Data Streams	Real-time streaming data ingestion engine capable of scaling shards on demand.
Compute / ETL Tier	AWS Lambda	Serverless event-driven processing function that transforms streaming records and writes to storage.
Data Lake Tier	Amazon S3	Scalable object storage structured into partitions (`/year=/month=/day=`) with KMS server-side encryption.
Storage Lifecycle	S3 Lifecycle Rules	Automated tiering policy transitioning raw objects to S3 Glacier Flexible / Deep Archive and enforcing expiration.
Catalog & Schema	AWS Glue Data Catalog	Central metadata repository storing table schemas discovered via automated Glue Crawlers.
Analytics Tier	Amazon Athena	Serverless interactive query engine using standard SQL

Component	AWS Service	Architectural Purpose
		directly against partitioned data in S3.

Description of the Project

In this lab, you will design and implement an end-to-end real-time analytics data pipeline:

1. **Streaming Data Ingestion:** Provision an Amazon Kinesis Data Stream to continuously receive raw JSON telemetry events (e.g., IoT metrics, user activity logs).
2. **Serverless Transformation & Partitioning:** Deploy an AWS Lambda function triggered by Kinesis shard batches to process payloads, enrich metadata, and store partitioned objects in Amazon S3.
3. **Storage Tiering & Lifecycle Management:** Configure Amazon S3 Lifecycle Policies to automatically transition data from S3 Standard to Standard-IA and Glacier Deep Archive to optimize long-term storage costs.
4. **Automated Schema Cataloging:** Deploy an AWS Glue Crawler to automatically discover schema structures and update the AWS Glue Data Catalog.
5. **Serverless Interactive Analytics:** Query the partitioned Data Lake using ANSI SQL in Amazon Athena, verifying partition pruning to minimize scanned bytes and query costs.

Step-by-Step Implementation Guide

Step 1: Create Amazon S3 Data Lake & Define Tiering Policies

1. **Navigate to Amazon S3 Console ➔ Create bucket:**
   - **Bucket Name:** `enterprise-data-lake-<your-account-id>`
   - **Encryption:** Enable Server-side encryption with AWS Key Management Service (SSE-KMS) or AWS-managed key (`aws/s3`).
   - **Versioning:** Enable Bucket Versioning.
2. **Configure Lifecycle Rules:**
   - Go to **Management ➔ Lifecycle rules ➔ Create lifecycle rule**.
   - **Rule Name:** `TieringAndExpirationPolicy`
   - **Filter:** Apply to all objects in the bucket.
   - **Transitions:**

        - Transition current versions of objects to **S3 Standard-IA** after 30 days.
        - Transition objects to **S3 Glacier Flexible Retrieval** or **Glacier Deep Archive** after 90 days.
    - **Expiration:** Set noncurrent versions/objects to expire/delete after 365 days.

Step 2: Provision Streaming Ingestion with Amazon Kinesis

1. **Navigate to Kinesis Console → Create Data Stream:**
    - **Data Stream Name:** `telemetry-data-stream`
    - **Capacity Mode:** Provisioned (1 Shard) or On-Demand.
2. **Test Stream Ingestion:**
    - Using AWS CloudShell or AWS CLI, test pushing sample records:aws kinesis put-record \

      --stream-name telemetry-data-stream \

      --partition-key sensor-1 \

      --data '{"device_id": "sensor-1", "temperature": 24.5, "timestamp": 1700000000}'

Step 3: Deploy AWS Lambda Serverless Transformation & Trigger

1. **Create IAM Role for Lambda:**
    - **Role Name:** `LambdaKinesisS3ExecutionRole`
    - **Permissions:** Attach `AWSLambdaKinesisExecutionRole` and grant `s3:PutObject` on `enterprise-data-lake-<your-account-id>`.
2. **Deploy Lambda Function:**
    - **Function Name:** `ProcessKinesisTelemetry` | **Runtime:** Python 3.12
    - **Execution Role:** `LambdaKinesisS3ExecutionRole`
    - Add a **Kinesis Trigger**: Source = `telemetry-data-stream`, Batch size = 100, Batch window = 60 seconds.

Step 4: Catalog Schemas with AWS Glue Crawler

1. **Navigate to AWS Glue Console → Crawlers → Create crawler:**
    - **Name:** `telemetry-s3-crawler`
    - **Data Source:** `s3://enterprise-data-lake-<your-account-id>/raw-telemetry/`
    - **IAM Role:** Create/select an IAM role with `AWSGlueServiceRole` and S3 read permissions.
    - **Target Database:** Create database `analytics_db`.
2. **Run Crawler:**

- Execute the crawler and verify that the table `raw_telemetry` is generated in the Glue Data Catalog.

Step 5: Execute Serverless SQL Queries in Amazon Athena

1. **Configure Athena Settings:**
   - Set **Query result location** to `s3://enterprise-data-lake-<your-account-id>/athena-results/`.
2. **Run Analytical Queries:**
   - Select database `analytics_db` and query the schema:SELECT device_id, COUNT(*) as event_count, AVG(temperature) as avg_temp

     FROM analytics_db.raw_telemetry

     GROUP BY device_id;

3. **Verify Cost & Performance:**
   - Observe "Data Scanned" metrics in Athena to verify partition pruning efficacy.

Create an S3 bucket

AWS Management | S3 buckets | S3 | us | Lifecycle configura

us-west-1.console.aws.amazon.com/s3/buckets?region=us-west-1

aws | United States | carminecompa33 (5860172... | carminecompa33

Amazon S3 > Buckets

Buckets

General purpose buckets | All AWS Regions | Directory buckets

General purpose buckets (7) Info

Copy ARN | Empty | Delete | Create bucket

Buckets are containers for data stored in S3.

data-lake | 1 match

1

Name	AWS Region	Creation date
enterprise-data-lake-586017285548	US West (N. California) us-west-1	October 20, 2026, 07:21:18 (UTC+02:00)

Account snapshot Info

Created a lifecycle for the bucket

Current version actions

Day 0

- Objects uploaded

Day 30

- Objects move to Standard-IA

Day 90

- Objects move to Glacier Flexible Retrieval (formerly Glacier)

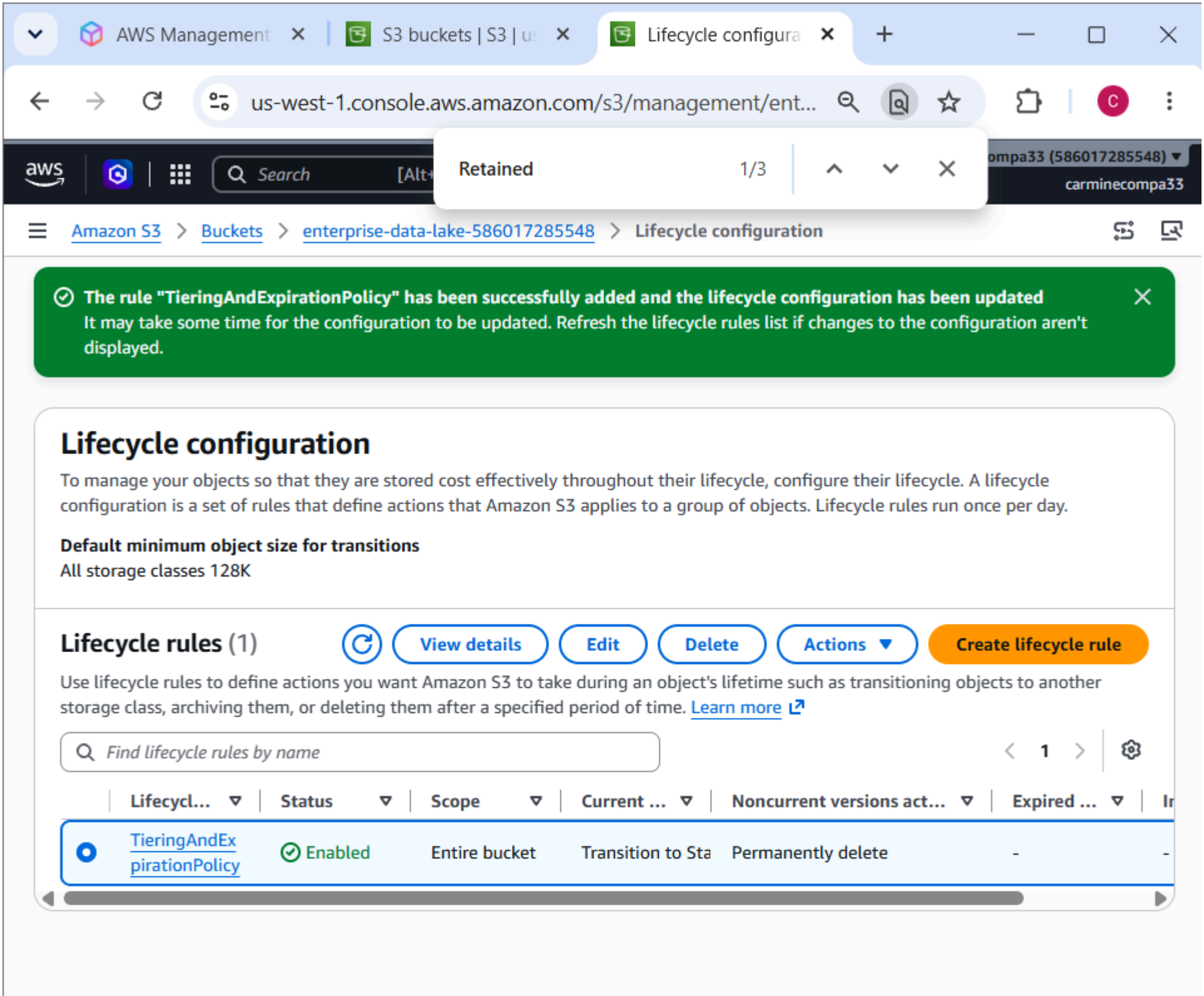
Noncurrent versions actions

Day 0

- Objects become noncurrent

Day 365

- 0 newest noncurrent versions are retained
- All other noncurrent versions are permanently deleted


AWS Management
S3 buckets | S3 | us
Lifecycle configura
us-west-1.console.aws.amazon.com/s3/management/ent...
Retained 1/3
aws
Search [Alt+
ompa33 (586017285548)
carminecompa33
Amazon S3 > Buckets > enterprise-data-lake-586017285548 > Lifecycle configuration
The rule "TieringAndExpirationPolicy" has been successfully added and the lifecycle configuration has been updated
It may take some time for the configuration to be updated. Refresh the lifecycle rules list if changes to the configuration aren't displayed.
Lifecycle configuration
To manage your objects so that they are stored cost effectively throughout their lifecycle, configure their lifecycle. A lifecycle configuration is a set of rules that define actions that Amazon S3 applies to a group of objects. Lifecycle rules run once per day.
Default minimum object size for transitions
All storage classes 128K
Lifecycle rules (1)
View details
Edit
Delete
Actions
Create lifecycle rule
Use lifecycle rules to define actions you want Amazon S3 to take during an object's lifetime such as transitioning objects to another storage class, archiving them, or deleting them after a specified period of time. Learn more
Find lifecycle rules by name
1
Lifecycl...
Status
Scope
Current ...
Noncurrent versions act...
Expired ...
TieringAndExpirationPolicy
Enabled
Entire bucket
Transition to Sta
Permanently delete
-
-

Created Amazon Kinesis data stream

Data stream telemetry-data-stream successfully created

telemetry-data-stream Info

Delete

Data stream summary

Status
Active

Capacity mode
On-demand

Data retention period
1 day

Maximum record size
1024 KiB

ARN
arn:aws:kinesis:us-west-1:586017285548:stream/telemetry-data-stream

Record distribution strategy
User set

Creation time
October 20, 2026 at 07:36 GMT+2

Applications | Monitoring | Configuration | Enhanced fan-out (0) | Data viewer | S3 Deli

Producers Info

Producers put records into Kinesis Data Streams.

Amazon Kinesis Agent
Use a stand-alone Java software application to send data to the stream. Learn more

View in GitHub

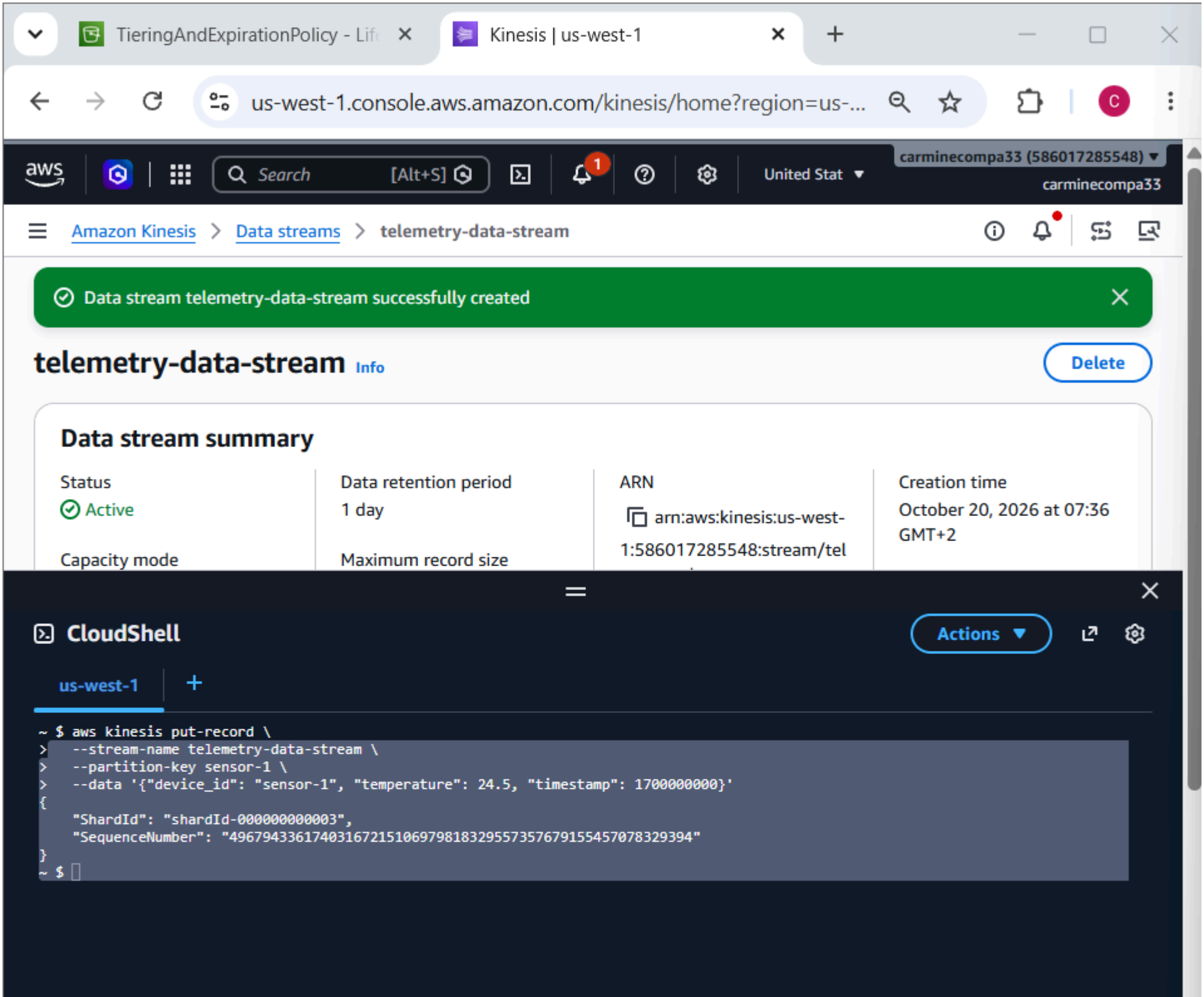
AWS SDK
Use AWS SDK for Java to develop producers. Learn more

View in GitHub

Amazon Kinesis Producer Library (KPL)
Use KPL to develop producers. Learn more

View in GitHub

Check that my Kinesis data stream is ready to accept data


TieringAndExpirationPolicy - Lif
Kinesis | us-west-1
us-west-1.console.aws.amazon.com/kinesis/home?region=us-...
Search
[Alt+S]
United Stat
carminecompa33 (586017285548)
carminecompa33
Amazon Kinesis
Data streams
telemetry-data-stream
Data stream telemetry-data-stream successfully created
telemetry-data-stream
Info
Delete
Data stream summary
Status
Active
Data retention period
1 day
ARN
arn:aws:kinesis:us-west-1:586017285548:stream/tel
Creation time
October 20, 2026 at 07:36 GMT+2
Capacity mode
Maximum record size
CloudShell
Actions
us-west-1
~ $ aws kinesis put-record \
> --stream-name telemetry-data-stream \
> --partition-key sensor-1 \
> --data '{"device_id": "sensor-1", "temperature": 24.5, "timestamp": 1700000000}'
{
"ShardId": "shardId-000000000003",
"SequenceNumber": "496794336174031672151069798183295573576791554570783293 94"
}
~ $

Created a role “LambdaKinesisS3ExecutionRole” for my Lambda function

TieringAndExpirati | Kinesis | us-west-1 | LambdaKinesisS3E

us-east-1.console.aws.amazon.com/iam/home?region=us-we...

aws | Search [Alt+S] | Globa | carminecompa33 (586017285548) | carminecompa33

IAM > Roles > LambdaKinesisS3ExecutionRole

Policy LambdaS3WritePolicy created.

LambdaKinesisS3ExecutionRole Info

Delete

Allows Lambda functions to call AWS services on your behalf.

Summary

Edit

Creation date
October 20, 2026, 07:47 (UTC+02:00)

ARN
arn:aws:iam::586017285548:role/LambdaKinesisS3ExecutionRole

Last activity
-

Maximum session duration
1 hour

Permissions | Trust relationships | Tags | Last Accessed | Revoke sessions

Permissions policies (2) Info

Simulate | Remove | Add permissions

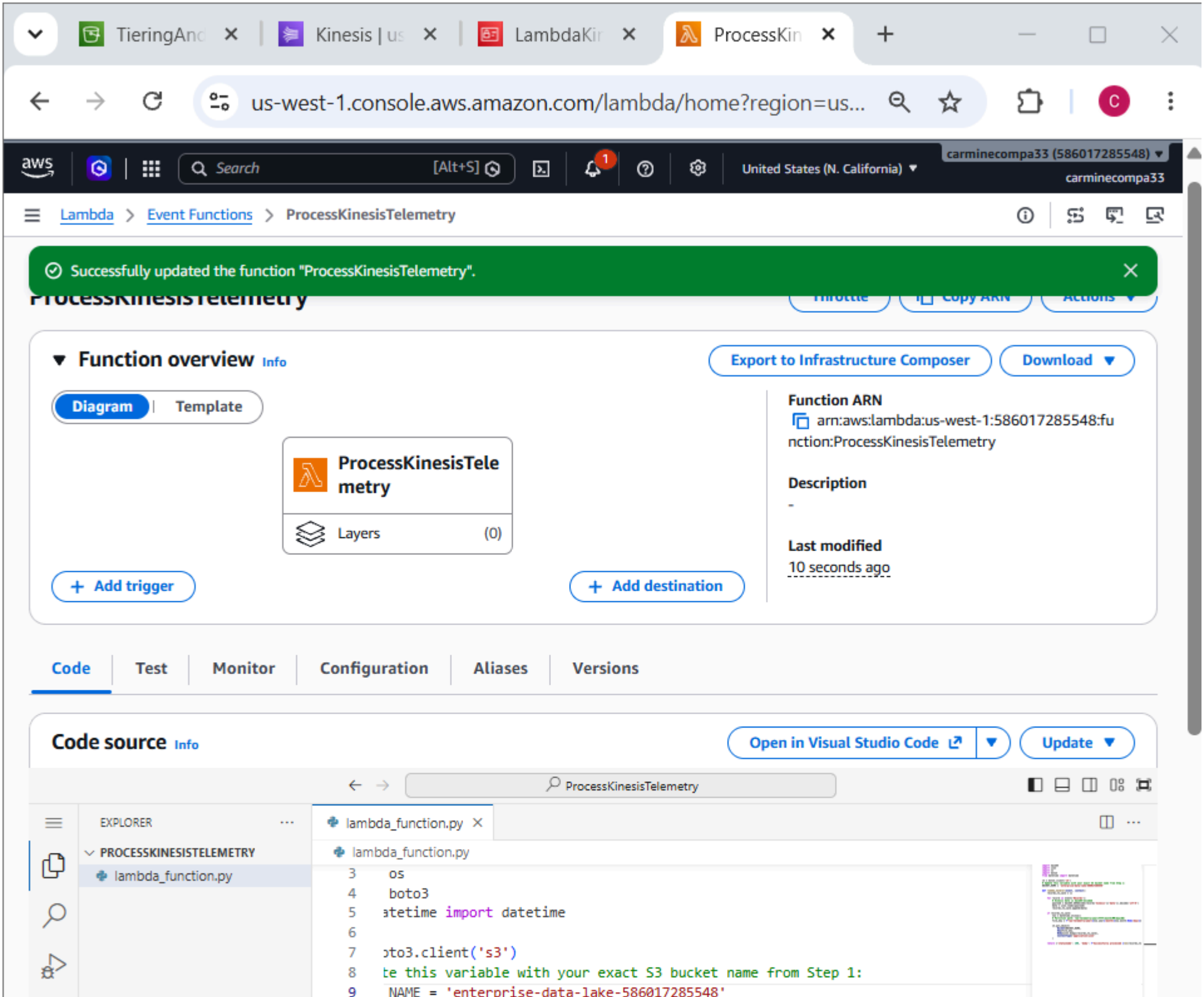
You can attach up to 20 managed policies.

Filter by Type

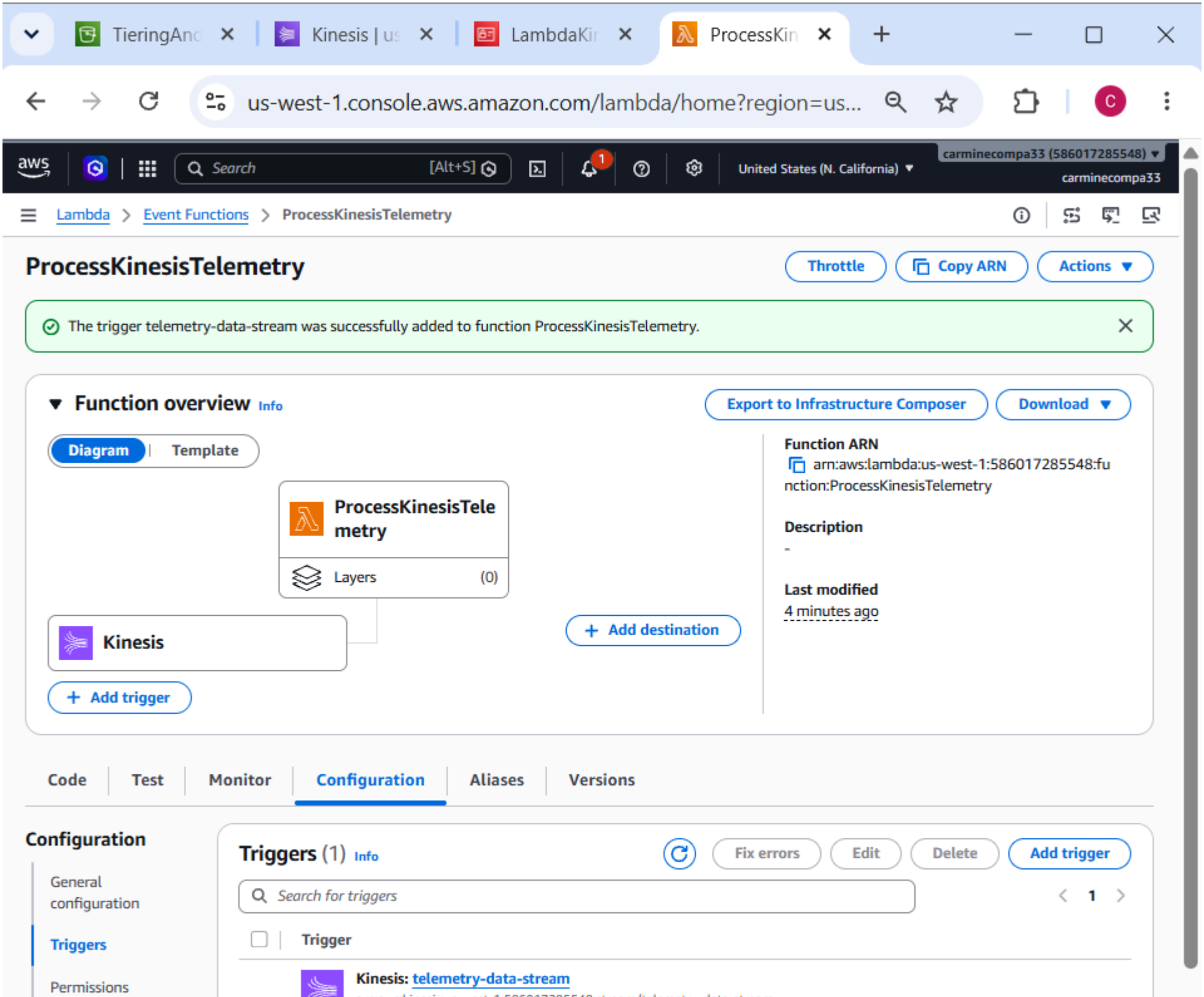
Search | All types | 1

Policy name	Type	Attached entities

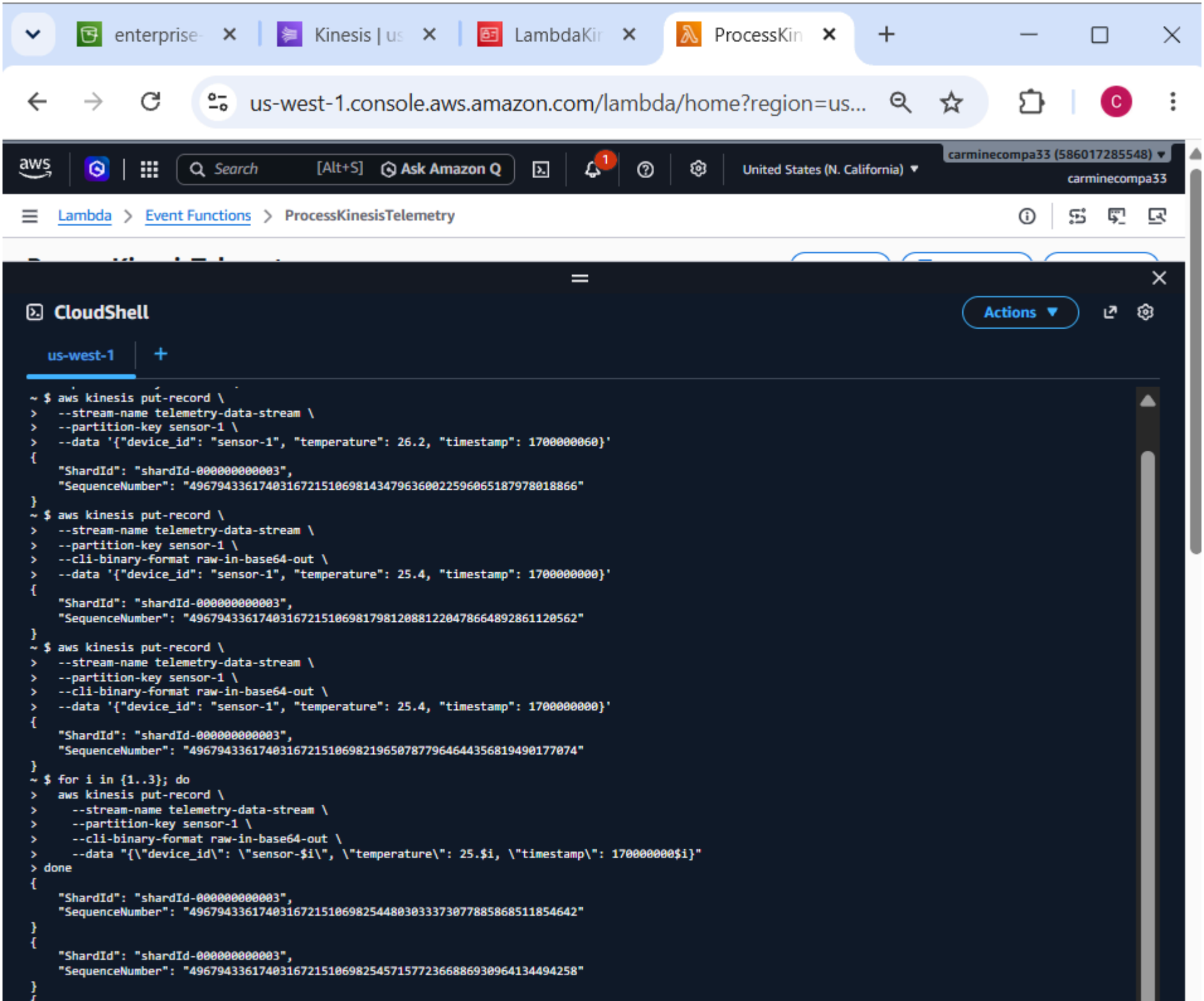
Created a Lambda function with the LambdaKinesisS3ExecutionRole


TieringAnc
Kinesis | us
LambdaKir
ProcessKin
us-west-1.console.aws.amazon.com/lambda/home?region=us...
aws
Search
[Alt+S]
United States (N. California)
carminecompa33 (586017285548)
carminecompa33
Lambda > Event Functions > ProcessKinesisTelemetry
Successfully updated the function "ProcessKinesisTelemetry".
Function overview Info
Export to Infrastructure Composer
Download
Diagram
Template
ProcessKinesisTelemetry
Layers (0)
Function ARN
arn:aws:lambda:us-west-1:586017285548:function:ProcessKinesisTelemetry
Description
-
Last modified
10 seconds ago
+ Add trigger
+ Add destination
Code
Test
Monitor
Configuration
Aliases
Versions
Code source Info
Open in Visual Studio Code
Update
ProcessKinesisTelemetry
EXPLORER
PROCESSKINESISTELEMETRY
lambda_function.py
lambda_function.py
os
boto3
atetime import datetime
oto3.client('s3')
te this variable with your exact S3 bucket name from Step 1:
NAME = 'enterprise-data-lake-586017285548'

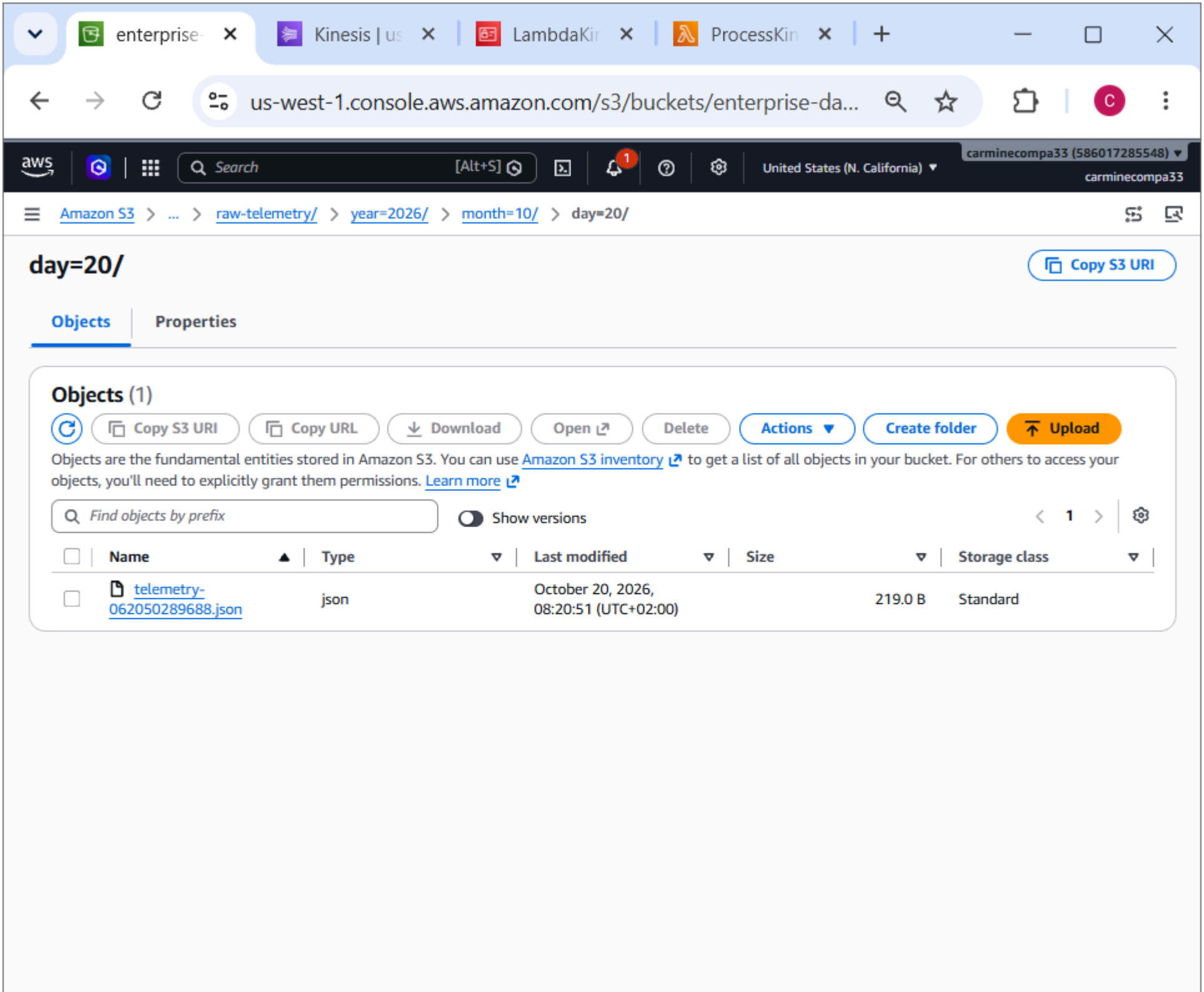
Attach a trigger to the Lambda function: Kinesis data stream with batch 100 and batch window 60 seconds


TieringAnc
Kinesis | us
LambdaKir
ProcessKin
us-west-1.console.aws.amazon.com/lambda/home?region=us...
aws
Search
[Alt+S]
United States (N. California)
carminecompa33 (586017285548)
carminecompa33
Lambda > Event Functions > ProcessKinesisTelemetry
ProcessKinesisTelemetry
Throttle
Copy ARN
Actions
The trigger telemetry-data-stream was successfully added to function ProcessKinesisTelemetry.
Function overview Info
Export to Infrastructure Composer
Download
Diagram
Template
ProcessKinesisTelemetry
Layers (0)
Kinesis
Add destination
Add trigger
Function ARN
arn:aws:lambda:us-west-1:586017285548:function:ProcessKinesisTelemetry
Description
-
Last modified
4 minutes ago
Code
Test
Monitor
Configuration
Aliases
Versions
Configuration
General configuration
Triggers
Permissions
Triggers (1) Info
Fix errors
Edit
Delete
Add trigger
Search for triggers
1
Trigger
Kinesis: telemetry-data-stream

Ingested data into Amazon Kinesis data strea,


enterprise
Kinesis | us
LambdaKir
ProcessKin
us-west-1.console.aws.amazon.com/lambda/home?region=us...
aws
Search
[Alt+S]
Ask Amazon Q
United States (N. California)
carminecompa33 (586017285548)
carminecompa33
Lambda
Event Functions
ProcessKinesisTelemetry
CloudShell
Actions
us-west-1
~ $ aws kinesis put-record \
> --stream-name telemetry-data-stream \
> --partition-key sensor-1 \
> --data '{"device_id": "sensor-1", "temperature": 26.2, "timestamp": 1700000060}'
{
"ShardId": "shardId-000000000003",
"SequenceNumber": "49679433617403167215106981434796360022596065187978018866"
}
~ $ aws kinesis put-record \
> --stream-name telemetry-data-stream \
> --partition-key sensor-1 \
> --cli-binary-format raw-in-base64-out \
> --data '{"device_id": "sensor-1", "temperature": 25.4, "timestamp": 1700000000}'
{
"ShardId": "shardId-000000000003",
"SequenceNumber": "49679433617403167215106981798120881220478664892861120562"
}
~ $ aws kinesis put-record \
> --stream-name telemetry-data-stream \
> --partition-key sensor-1 \
> --cli-binary-format raw-in-base64-out \
> --data '{"device_id": "sensor-1", "temperature": 25.4, "timestamp": 1700000000}'
{
"ShardId": "shardId-000000000003",
"SequenceNumber": "49679433617403167215106982196507877964644356819490177074"
}
~ $ for i in {1..3}; do
> aws kinesis put-record \
> --stream-name telemetry-data-stream \
> --partition-key sensor-1 \
> --cli-binary-format raw-in-base64-out \
> --data "{\"device_id\": \"sensor-$i\", \"temperature\": 25.$i, \"timestamp\": 170000000$i}"
> done
{
"ShardId": "shardId-000000000003",
"SequenceNumber": "49679433617403167215106982544803033373077885868511854642"
}
{
"ShardId": "shardId-000000000003",
"SequenceNumber": "49679433617403167215106982545715772366886930964134494258"
}

Lambda received Kinesis data stream and sent them to the S3 bucket


enterprise-
Kinesis | us
LambdaKir
ProcessKin
us-west-1.console.aws.amazon.com/s3/buckets/enterprise-da...
aws
Search
[Alt+S]
United States (N. California)
carminecompa33 (586017285548)
carminecompa33
Amazon S3 > ... > raw-telemetry/ > year=2026/ > month=10/ > day=20/
day=20/
Copy S3 URI
Objects
Properties
Objects (1)
Copy S3 URI
Copy URL
Download
Open
Delete
Actions
Create folder
Upload
Objects are the fundamental entities stored in Amazon S3. You can use Amazon S3 inventory to get a list of all objects in your bucket. For others to access your objects, you'll need to explicitly grant them permissions. Learn more
Find objects by prefix
Show versions
1
Name
Type
Last modified
Size
Storage class
telemetry-062050289688.json
json
October 20, 2026, 08:20:51 (UTC+02:00)
219.0 B
Standard

Below the code used in Lambda function:

```
import base64
import json
import os
import boto3
from datetime import datetime

s3 = boto3.client('s3')
BUCKET_NAME = 'enterprise-data-lake-586017285548'

def lambda_handler(event, context):
    print(f"Received event with {len(event.get('Records', []))} records.")
    records_to_save = []

    for record in event.get('Records', []):
        raw_data = record['kinesis']['data']

        # Handle string or bytes
        if isinstance(raw_data, str):
            try:
                payload_bytes = base64.b64decode(raw_data)
                payload_str = payload_bytes.decode('utf-8')
            except Exception:
                payload_str = raw_data
        else:
            payload_str = raw_data.decode('utf-8', errors='ignore')

        try:
            data = json.loads(payload_str)
            records_to_save.append(data)
            print(f"Successfully parsed payload: {data}")
        except Exception as e:
            print(f"Failed to parse JSON from payload: {payload_str}, error: {e}")

    if records_to_save:
```

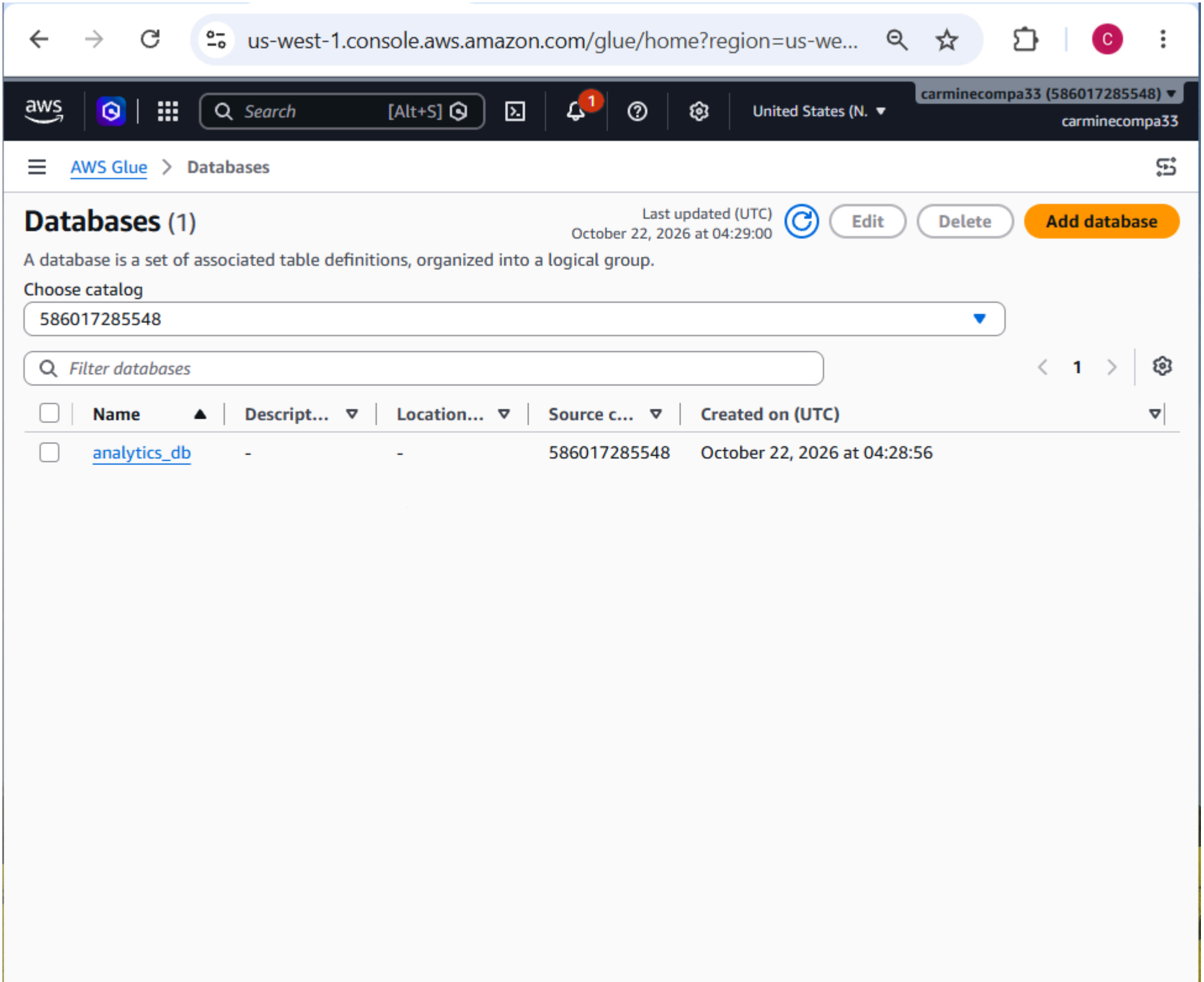
```
        now = datetime.utcnow()
        file_key = f"raw-telemetry/year={now.year}/month={now.month:02d}/day={now.day:02d}/telemetry-{now.strftime('%H%M%S%f')}.json"

        print(f"Writing {len(records_to_save)} records to S3 bucket {BUCKET_NAME} at key {file_key}")

        s3.put_object(
            Bucket=BUCKET_NAME,
            Key=file_key,
            Body=json.dumps(records_to_save),
            ContentType='application/json'
        )
        print("S3 PutObject successful!")
    else:
        print("No valid records to save in this invocation.")

    return {'statusCode': 200, 'body': f"Processed {len(records_to_save)} records."}
```

Created AWS Glue database


us-west-1.console.aws.amazon.com/glue/home?region=us-we...
aws
Search
[Alt+S]
United States (N.
carminecompa33 (586017285548)
carminecompa33
AWS Glue
Databases
Databases (1)
Last updated (UTC)
October 22, 2026 at 04:29:00
Edit
Delete
Add database
A database is a set of associated table definitions, organized into a logical group.
Choose catalog
586017285548
Filter databases
1
Name
Descript...
Location...
Source c...
Created on (UTC)
analytics_db
-
-
586017285548
October 22, 2026 at 04:28:56

Created AWS Glue crawler with

- AWSGlueServiceRole that has permission on the S3 bucket enterprise-data-lake-586017285548
- Target database: the new created AWS Glue analytics_db

Crawlers - / | Databases | enterprise- | New Tab

us-west-1.console.aws.amazon.com/glue/home?region=us-we...

aws | Search [Alt+S] | United States | carminecompa33 (586017285548) | carminecompa33

AWS Glue > Crawlers > telemetry-s3-crawler

One crawler successfully created
The following crawler is now created: "telemetry-s3-crawler"

telemetry-s3-crawler

Last updated (UTC)
October 22, 2026 at 04:30:39

Run crawler | Edit | Delete

Crawler properties

Name	IAM role	Database	State
telemetry-s3-crawler	AWSGlueServiceRole	analytics_db	READY

Description	Security configuration	Lake Formation configuration	Table prefix
-	-	-	-

Maximum table threshold
-

▶ Advanced settings

Crawler runs | Schedule | Data sources | Classifiers | Tags

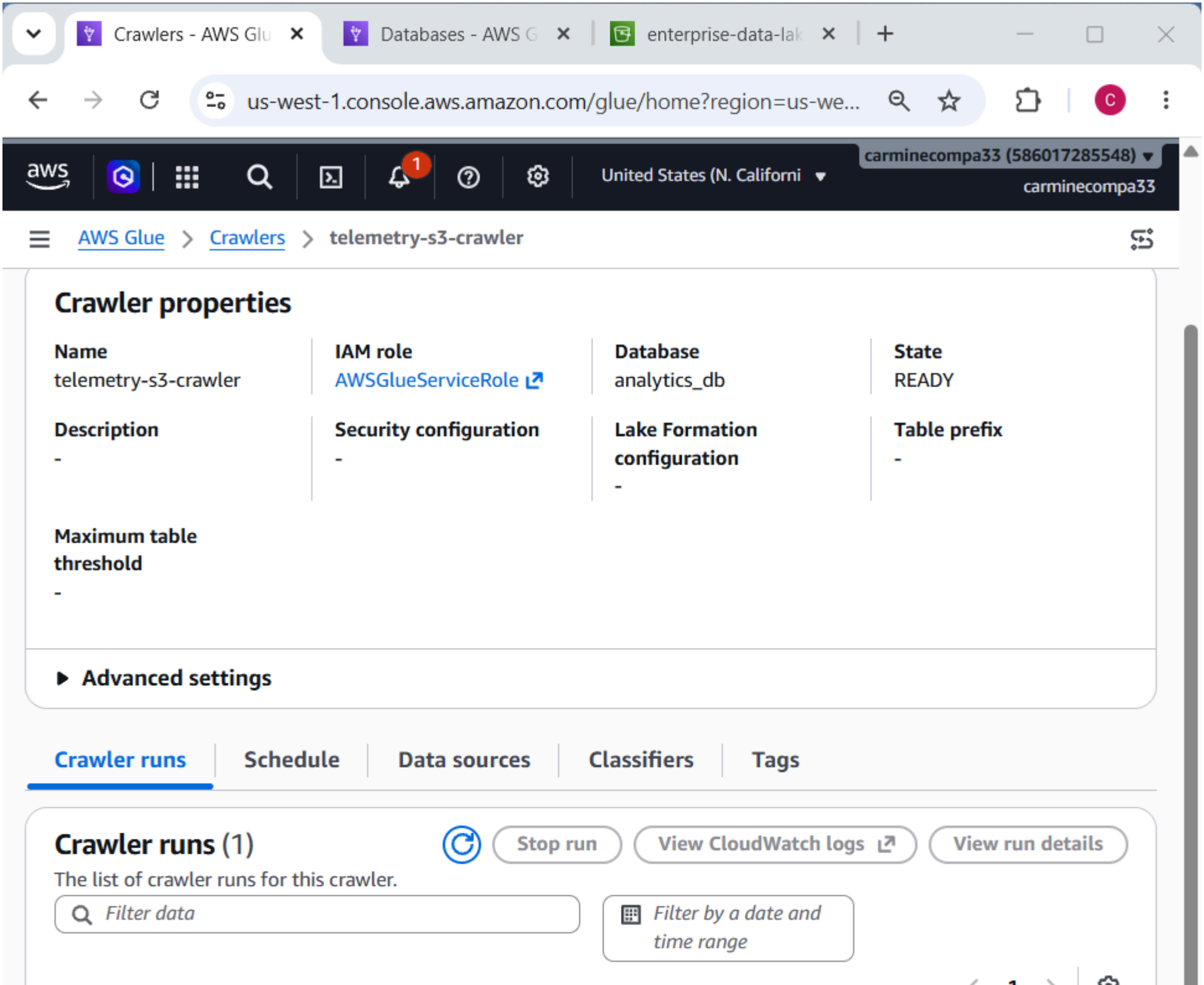
Crawler runs (0)

Stop run | View CloudWatch logs | View run details

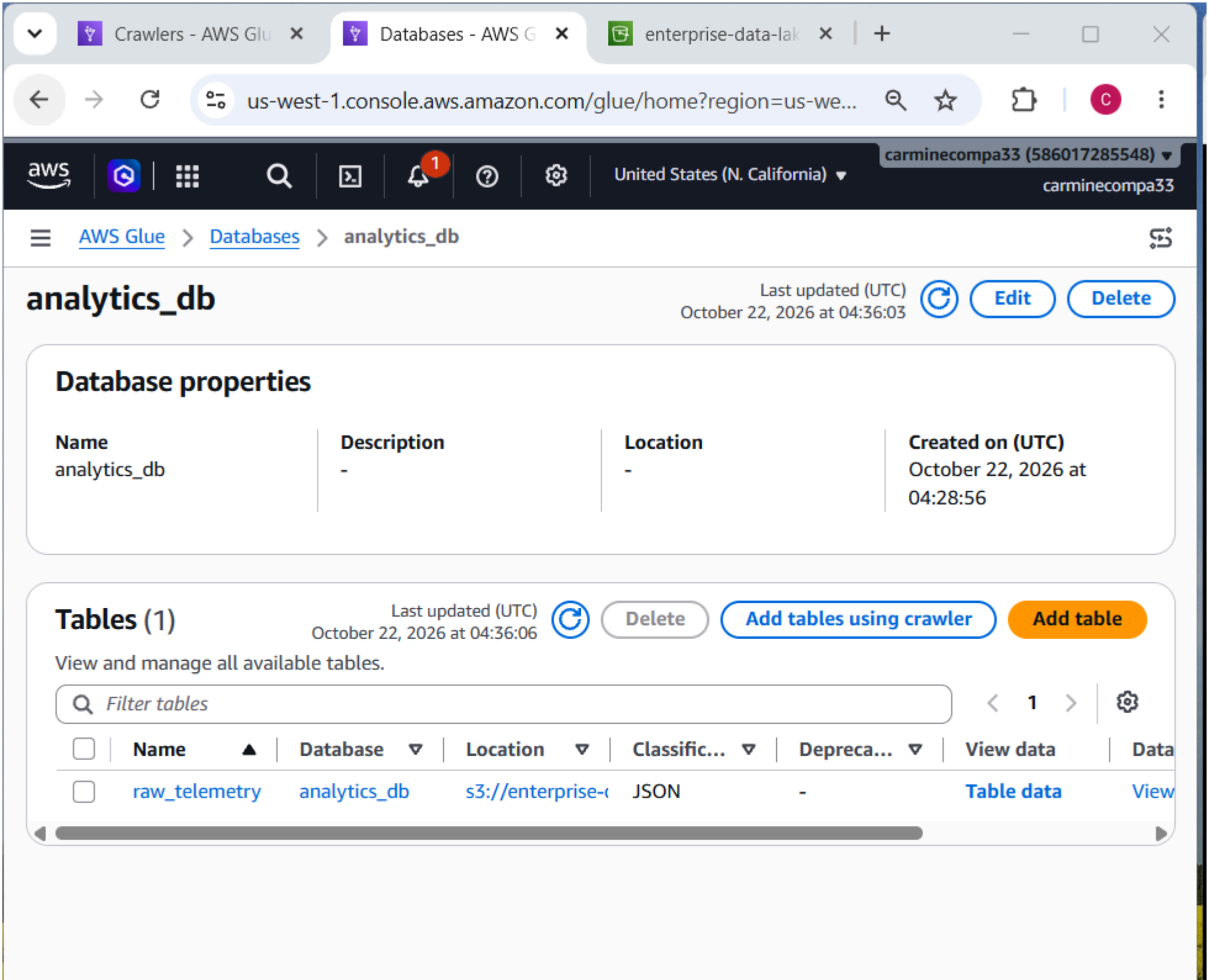
The list of crawler runs for this crawler.

Filter data | Filter by a date and time range

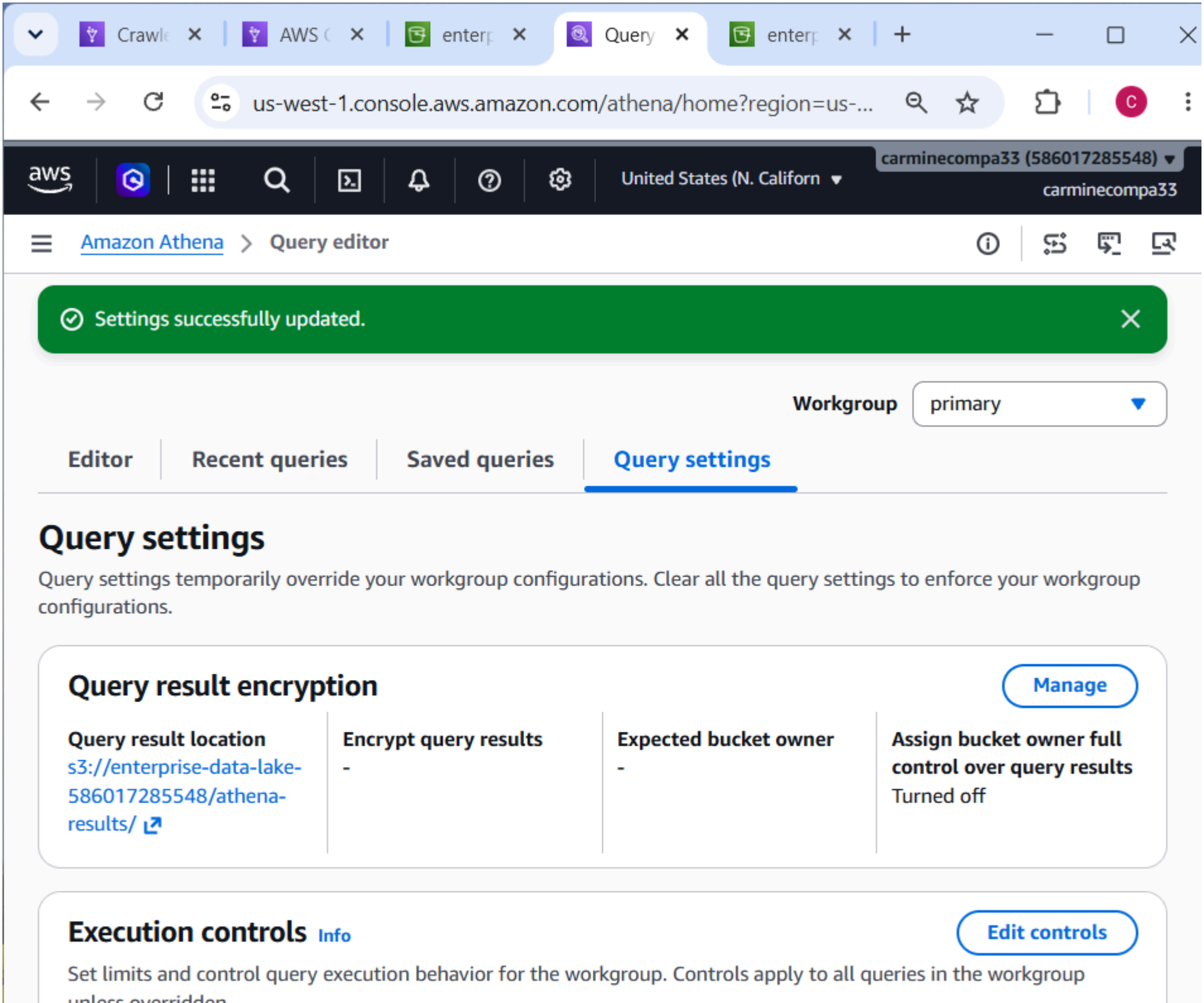
I have run the AWS Glue crawler on the S3 bucket


Crawlers - AWS Glu
Databases - AWS G
enterprise-data-lak
us-west-1.console.aws.amazon.com/glue/home?region=us-we...
aws
United States (N. Californi
carminecompa33 (586017285548)
carminecompa33
AWS Glue > Crawlers > telemetry-s3-crawler
Crawler properties
Name
telemetry-s3-crawler
IAM role
AWSGlueServiceRole
Database
analytics_db
State
READY
Description
-
Security configuration
-
Lake Formation configuration
-
Table prefix
-
Maximum table threshold
-
Advanced settings
Crawler runs
Schedule
Data sources
Classifiers
Tags
Crawler runs (1)
Stop run
View CloudWatch logs
View run details
The list of crawler runs for this crawler.
Filter data
Filter by a date and time range

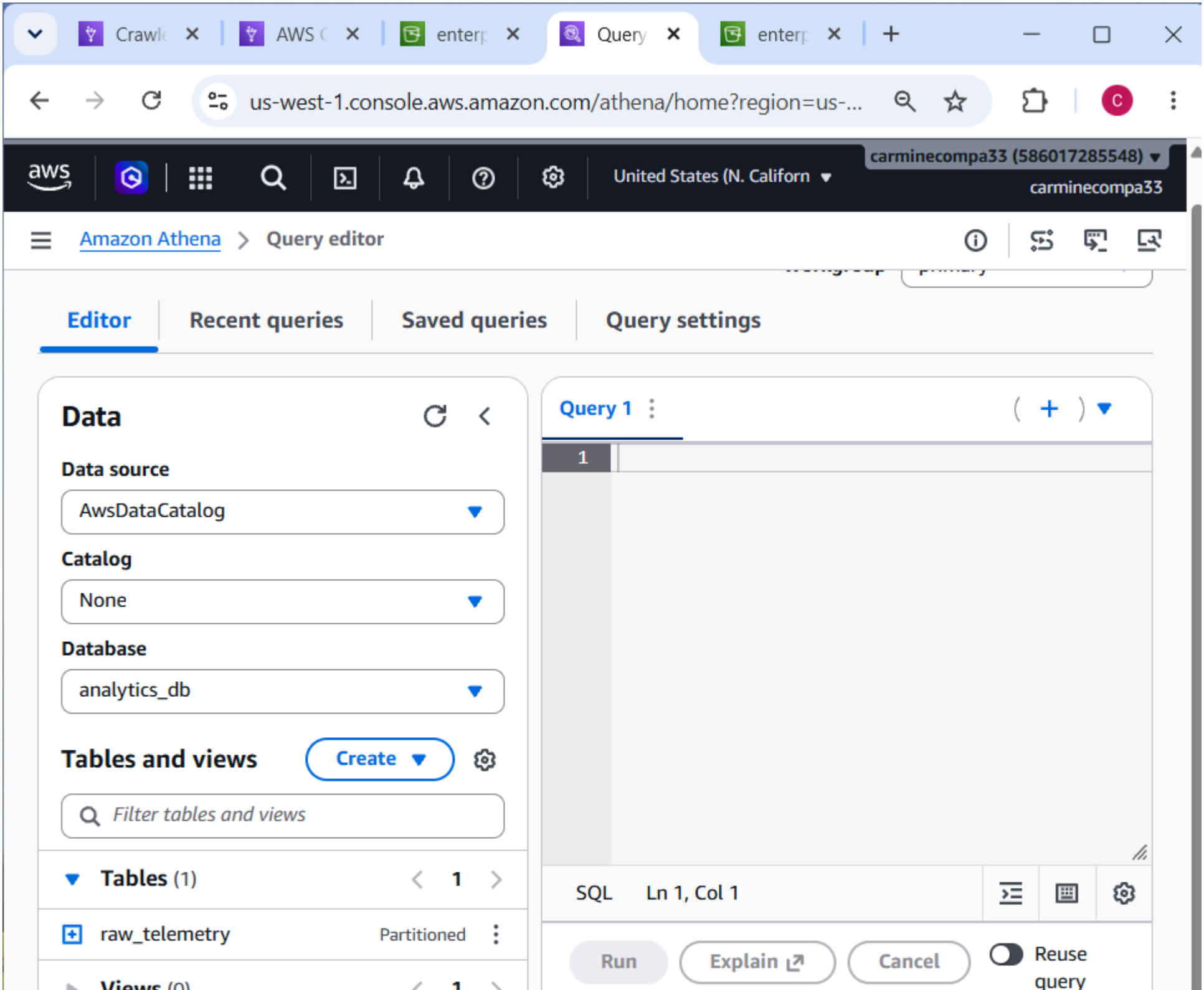
Now I can see the table created in AWS Glue target database analytics_db


Crawlers - AWS Glu
Databases - AWS G
enterprise-data-lak
us-west-1.console.aws.amazon.com/glue/home?region=us-we...
aws
United States (N. California)
carminecompa33 (586017285548)
carminecompa33
AWS Glue > Databases > analytics_db
analytics_db
Last updated (UTC)
October 22, 2026 at 04:36:03
Edit
Delete
Database properties
Name
analytics_db
Description
-
Location
-
Created on (UTC)
October 22, 2026 at 04:28:56
Tables (1)
Last updated (UTC)
October 22, 2026 at 04:36:06
Delete
Add tables using crawler
Add table
View and manage all available tables.
Filter tables
1
Name
Database
Location
Classific...
Depreca...
View data
Data
raw_telemetry
analytics_db
s3://enterprise-(
JSON
-
Table data
View

Opened Amazon Athena and added the folder “athena_results” in the target of the Amazon Athena SQL queries output

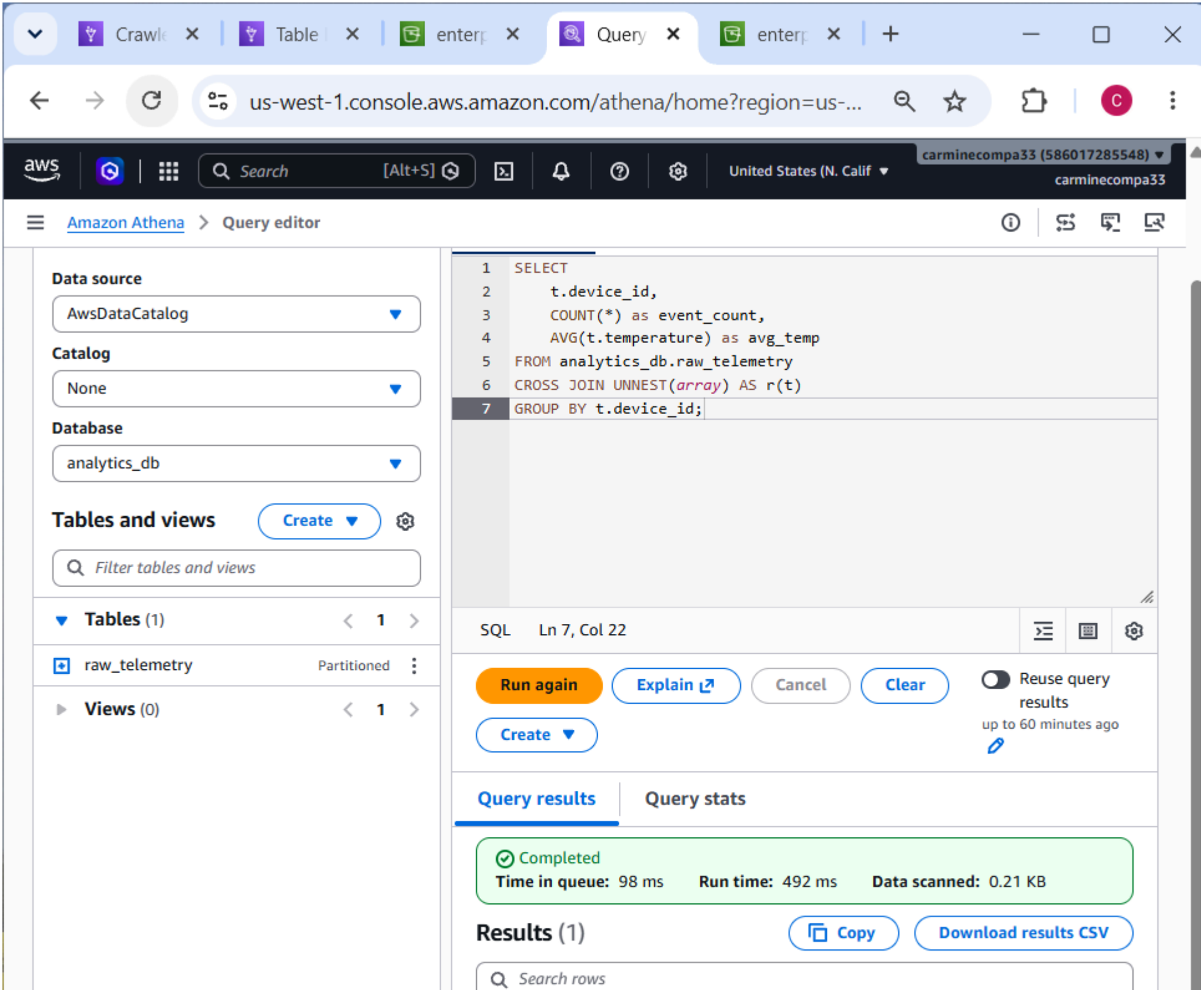

Crawle
AWS C
enterp
Query
enterp
us-west-1.console.aws.amazon.com/athena/home?region=us-...
aws
United States (N. Californ
carminecompa33 (586017285548)
carminecompa33
Amazon Athena
Query editor
Settings successfully updated.
Workgroup
primary
Editor
Recent queries
Saved queries
Query settings
Query settings
Query settings temporarily override your workgroup configurations. Clear all the query settings to enforce your workgroup configurations.
Query result encryption
Manage
Query result location
s3://enterprise-data-lake-586017285548/athena-results/
Encrypt query results
-
Expected bucket owner
-
Assign bucket owner full control over query results
Turned off
Execution controls
Info
Edit controls
Set limits and control query execution behavior for the workgroup. Controls apply to all queries in the workgroup

In Amazon Athena >> Editor >> Data >> I can see the analytics_db AWS Glue database and the related table “raw_telemetry” created before when we ran AWS Glue crawler on the S3 bucket


Crawle
AWS (
enterp
Query
enterp
us-west-1.console.aws.amazon.com/athena/home?region=us-...
aws
United States (N. Californ
carminecompa33 (586017285548)
carminecompa33
Amazon Athena
Query editor
Editor
Recent queries
Saved queries
Query settings
Data
Data source
AwsDataCatalog
Catalog
None
Database
analytics_db
Tables and views
Create
Filter tables and views
Tables (1)
1
raw_telemetry
Partitioned
Query 1
1
SQL
Ln 1, Col 1
Run
Explain
Cancel
Reuse
query

I run SQL queries on the Amazon Athena SQL editor against the analytics_db and it works.
Example:

```
SELECT
   t.device_id,
   COUNT(*) as event_count,
   AVG(t.temperature) as avg_temp
FROM analytics_db.raw_telemetry
CROSS JOIN UNNEST(array) AS r(t)
GROUP BY t.device_id;
```


Crawl
Table
enterp
Query
enterp
us-west-1.console.aws.amazon.com/athena/home?region=us-...
aws
Search
[Alt+S]
United States (N. Calif
carminecompa33 (586017285548)
carminecompa33
Amazon Athena
Query editor
Data source
AwsDataCatalog
Catalog
None
Database
analytics_db
Tables and views
Create
Filter tables and views
Tables (1)
raw_telemetry
Partitioned
Views (0)
1 SELECT
2 t.device_id,
3 COUNT(*) as event_count,
4 AVG(t.temperature) as avg_temp
5 FROM analytics_db.raw_telemetry
6 CROSS JOIN UNNEST(array) AS r(t)
7 GROUP BY t.device_id;
SQL Ln 7, Col 22
Run again
Explain
Cancel
Clear
Create
Reuse query results
up to 60 minutes ago
Query results
Query stats
Completed
Time in queue: 98 ms
Run time: 492 ms
Data scanned: 0.21 KB
Results (1)
Copy
Download results CSV
Search rows

Lab Completion Summary

1. **S3 Lake & Lifecycle:** Provisioned bucket with tiering policies (Standard-IA → Glacier).
2. **Kinesis Streaming:** Created the ingestion stream to handle telemetry records.
3. **Lambda ETL:** Processed real-time stream batches and partitioned raw JSON data in S3.
4. **AWS Glue Catalog:** Discovered and registered table schemas in analytics_db.
5. **Athena Analytics:** Successfully queried partitioned data using ANSI SQL.